4.1 Allocating Memory

After Sam finished his image program involving a multi-dimensional array, something was bothering him. While the program worked with the current 12 megapixel camera he owns, it will not work with images of any other size. This struck him as short-sighted; an image program should be able to work with any size image, even image sizes not known at compile time. In an effort to work around this glaring shortcoming, he discovers memory allocation.

Objectives

By the end of this class, you will be able to:

- Allocate memory with the `new` operator.
- Free allocated memory with the `delete` operator.
- Allocate and free single and multi-dimensional arrays.

Prerequisites

Before reading this section, please make sure you are able to:

- Declare a pointer variable (Chapter 3.3).
- Get the data out of a pointer (Chapter 3.3).
- Pass a pointer to a function (Chapter 3.3).

Overview

Dynamic memory allocation is the process of a program reserving an amount of memory that is known at runtime rather than at compile time. In other words, the program is able to reserve as much memory as the user requires, even when the programmer has no idea how much memory that will be.

This is best explained by an analogy. Imagine a developer wishing to purchase an acre of land. He goes to City Hall to acquire two things: a deed and the address of the land. The deed is a guarantee the land will not be developed by anyone else, and the address is a pointer to the land so he knows where to find it. If there is no land available, then he will have to deal with the setback and make other plans. If land is available, the developer will walk out with a valid address. With this address and deed, the developer goes off to do something useful and productive with the newly acquired acre. Of course, the acre is not clean. The previous inhabitant of the land left some landscaping and structures on the land which will need to be removed and leveled before any building occurs. The developer retains ownership of the land until his business is completed. At this time, he returns to City Hall and returns the deed and forgets the address of the land.

This process is exactly what happens when working with memory allocation. The program (developer) asks the operating system (City Hall) for a range of memory (acre of land). If the request is greater than the amount of available memory, the request returns a failure condition which the program will need to handle. Otherwise, a pointer to the memory (address) will be given to the program as well as a guarantee that no other program will be given the same memory (deed). This memory is filled with random 1's and 0's from the previous occupant (landscaping and structures on the land) which will need to be initialized (leveled). When the program is finished with the memory, it should be returned to the operating system (returns the deed to City Hall) and the pointer should be set to `NULL` (forgets the address).

There are three parts to this process:

- `NULL`: The empty address indicating a pointer is invalid
- `new`: The operator used to request memory from the operating system
- `delete`: The operator used to tell the operating system the memory is no longer needed

NULL Pointer

Up until this point, all the pointers we have used in our programs pointed to an existing location in memory. This location was always a local variable, meaning we could always assume that the pointer is referring to a valid location in memory. However, there often arises the occasion when the pointer refers to nothing. This situation requires us to mark the pointer so we can tell by inspection whether the address is valid.

To illustrate this point, remember our assignment (Assignment 3.2) where we computed the student's grade. The important thing about this assignment is that we are not to factor in the assignments the student has yet to complete. We marked these assignments with a `-1` score. In essence, the `-1` is a special token indicating the score is invalid or not yet completed. Thus, by inspection, we can tell if a score is invalid:

```
if (scores[i] == -1)
   cout << "No score for assignment " << i << endl;
```

The `NULL` pointer is essentially the same thing: an indication that a given pointer refers to no location in memory. We can check the validity of a pointer with a "`NULL`-check:"

```
if (p == NULL)
    cout << "The pointer does not refer to a valid location in memory\n";
```

Definition of NULL

The first thing to realize about `NULL` is that it is an address. While we have created pointers to characters and pointers to integers, `NULL` is a pointer to `void`. This means we can assign `NULL` to any pointer without error:

```
{
   int   * pGrade   = NULL;          // we can assign NULL to any
   float * pAccount = NULL;          //    type of pointer without
   char  * name     = NULL;          //    casting
}
```

The second thing to realize about `NULL` is that the numeric address is zero. Thus, the definition of `NULL` is:

```
#define NULL (void *)0x00000000
```

As you can well imagine, choosing zero as the `NULL` address was done on purpose. All valid memory locations are guaranteed to be not zero (the operating system owns that location: the first instruction to be executed when a computer boots). Also, zero is the only `false` value so `NULL` is the only `false` address. This makes doing a "`NULL`-check" easy:

```
if (p)                                    // same as "if (p != NULL)"
   cout << "The address of p is valid!\n";
```

Using NULL

One use of the NULL address is to indicate that a pointer is not valid. This can be done when there is nothing to point to. Consider the following function displaying the highest 'A' in a list of numeric grades. While commonly there is at least one 'A' in a list of student grades, it is not always the case.

```
/**********************************
 * DISPLAY HIGHEST A
 * Given a list of numeric grades
 * for a class, display the highest
 * A if one exists
 **********************************/
void displayHighestA(const int grades[],       // we won't be changing this so
                     int num)                  //    the array is a const
{
   const int * pHighestA = NULL;               // Initially no 'A's were found

   // find the highest A
   for (int count = 0; count < num; count++)   // loop through all the grades
      if (grades[count] >= 90)                 // only A's please
      {
         if (pHighestA == NULL)                // if none were found, then any
            pHighestA = &(grades[count]);      //     'A' is the highest
         else if (*pHighestA < grades[count])  // otherwise, only the highest if it
            pHighestA = &(grades[count]);      //     is better than any other
      }

   // output the highest A
   if (pHighestA)                              // classic NULL check: only display the
      cout << *pHighestA << endl;              //     'A' if one was found
   else                                        // otherwise (pHighestA == NULL),
      cout << "There was not an A\n";          //     none was found
}
```

NULL check

Probably the most common use of NULL is to do a "NULL-check." Because we expect our program to be set up correctly and pointers to always have valid addresses, it is common to add an assert just before dereferencing a pointer to make sure we won't crash.

```
{
   char * pLetter = NULL;                      // first set the pointer to NULL
                                               //     to indicate it is uninitialized
   if (isalpha(value[0]))
      pLetter = value + 0;                     // observe how pLetter is set in both
   else                                        //     conditions of the IF statement
      pLetter = value + 1;

   assert(pLetter != NULL);                    // like any assert, this should never
   cout << "Letter: "                          //     fire unless the programmer made
        << *pLetter                            //     a mistake. It is better to fire
        << endl;                               //     than to crash, of course!
   pLetter = NULL;                             // indicate we are done by setting the
}                                              //     pointer back to NULL
```

If we habitually set our pointers to NULL and then assert just before they are dereferenced, we can catch a ton of bugs. These bugs are also easy to fix, of course; much easier than a random crash!

Allocation with New

When we declare a local variable (also known as a stack variable), the compiler takes care of memory management. The compiler makes sure that there is memory reserved for the variable as soon as the variable falls into scope. The compiler also makes sure the memory is freed as soon as the variable falls out of scope. While this is very convenient, it can also be very limiting: the compiler needs to know the size of the block of memory to be reserved and how long it will be needed. Memory allocation relaxes both of these constraints.

We request new memory from the operating system with the `new` operator. The syntax is:

```
<pointer variable> = new <data-type>;
```

If, for example, a `double` is to be allocated, it is accomplished with:

```
{
   double * p;                // "p" is a pointer to a double
   p = new double;            // allocating a double returns a pointer to a double
}
```

We can also initialize a block of memory at allocation time. The syntax is very similar:

```
<pointer variable> = new <data-type> ( <initialization value> );
```

If, for example, you wish to allocate a character and initialize it with the letter 'A', then:

```
{
   char * p = new char('A');
}
```

This does three things: it reserves a byte of memory (`sizeof(char) == 1`), it initializes that value to 65 (`'A' == 65`), and it sends that address to the variable `p`.

Memory allocation failure

We cannot generally assume that a memory allocation is successful. In other words, it might be the case that there is no more memory to be had. Our code needs to be able to detect this condition and gracefully handle the error.

When a `new` request fails, the resulting pointer is `NULL`. However, we need to tell `new` that we wish to be notified of a failure in terms of the `NULL` pointer. This is done with the `nothrow` parameter:

```
{
   int * p = new (nothrow) int;                    // notice the nothrow parameter
   if (p == NULL)                                  // failure comes in the form of a
      cout << "Memory allocation failure!\n";      //     NULL pointer
}
```

Every memory allocation should be accompanied by a `NULL` check; never assume an allocation succeeded.

Sam's Corner

There are two ways the new operator reports errors: returning a `NULL` pointer or throwing an exception. Since exception handling is a CS 165 topic, we will use the `NULL` check this semester.

Allocating arrays

We allocate arrays much the same way we allocate individual data-types. The difference, of course, is that we need to tell new how many instances of the data-type are needed. The syntax is:

```
<array variable> = new <data-type> [ <size> ];
```

Note that, unlike with array local variables, the size parameter does not need to be a constant or a literal. In other words, since `new` is essentially a function call, the compiler does not need to know how much data will be allocated; it can be determined at run-time. Consider the following code:

```
{
   // get the size of the text
   int size;                                        // memory size variables are integers
   do
   {
      cout << "How long is your name? ";
      cin  >> size;                                 // continue prompting until the
   }                                                //    user gives us a positive
   while (size <= 0);                               //    size

   // allocate the memory
   char * text = new(nothrow) char [size + 1];      // allocate one more for \0
   if (!text)                                       // same as "if (text != NULL)"
      cout << "No memory! This is bad!\n";          // should return because we will
                                                    //    crash in a minute…
   // prompt for the name
   cout << "What is your name? ";
   cin.getline(text, size + 1);                     // treat "text" like any other string
}
```

Freeing with Delete

Once we are finished with a given block of memory, it is important to return it to the operating system so another program (or part of our own program!) can use it. This is accomplished with the `delete` operator. Note that we don't need to do this with traditional local variables because, once the variable falls out of scope, the compiler frees it for us. However, with memory allocation, the programmer (not the compiler!) indicates when the memory is no longer needed.

The syntax for the `delete` operator is:

```
delete <pointer variable>;
delete [] <array pointer variable>;
```

Consider the following example to allocate an integer and a string:

```
{
   int  * p = new int;              // allocate 4 bytes
   char * text = new char[256];      // allocate 256 bytes

   delete p;                        // no []s to free a single slot in memory
   delete [] text;                  // the []s indicate an array is freed.
}
```

Sue's Tips

To make sure we don't try to use newly freed memory, always assign the pointer to `NULL` after `delete`.

Example 4.1 – AllocateValue

Demo

In the past, we used pointers to refer to data that was declared elsewhere. In the following example, the pointer is referring to memory we newly allocated: a `float`. We will allocate space for a variable, fill the variable with a value, and free the memory when completed.

Solution

There are four parts to this process: creating a pointer variable so we can remember the memory location that was allocated, allocate the memory with `new`, use the memory location using the dereference operator `*`, and freeing the memory with `delete` when finished.

```
/**********************************
 * EXAMPLE
 * This is a bit contrived so I can't think
 * of a better name
 **********************************/
void example()
{
   // At first, the pointer refers to nothing. We use the NULL pointer
   // to signify the address is invalid or uninitialized
   float * pNumber = NULL;

   // now we will allocate 4 bytes for a float.
   pNumber = new(nothrow) float;
   if (!pNumber)
      return;

   // at this point (no pun intended), we can use it like any other pointer
   assert(pNumber);
   *pNumber = 3.14159;

   // Regular variables get recycled after they fall out of scopes. Not true
   // with allocated data. We need to free it with delete
   delete pNumber;
   pNumber = NULL;
}
```

Challenge

As a challenge, try to break this example into three functions: one function to allocate the memory returning a pointer to a float, one to change the value taking a pointer to a float as a parameter, and a final function to free the data.

See Also

The complete solution is available at 4-1-allocateValue.cpp or:

```
/home/cs124/examples/4-1-allocateValue.cpp
```

Example 4.1 – Allocate Array

Demo

This example will demonstrate how to allocate an array of integers. Unlike with traditional arrays, we will be able to prompt the user for the number of items in the array.

Problem

Write a program to prompt the user for the number of items in a list and the values for the list.

```
How many items? 3
Please enter 3 values
        # 1: 100
        # 2: 200
        # 3: 400
A display of the list:
        100
        200
        400
```

Solution

First, we will write a function to allocate the list given, as a parameter, the number of items.

```
int * allocate(int numItems)
{
   assert(numItems > 0);                      // better be a positive number!
   // Allocate the necessary memory
   int *p = new(nothrow) int[numItems];       // all the work is done here.

   // if p == NULL, we failed to allocate
   if (!p)
      cout << "Unable to allocate " << numItems * sizeof(int) << " bytes\n";
   return p;
}
```

This function is called by `main()`, which also calls a function to fill and display the list.

```
int main()
{
   int numItems = getNumItems();
   assert(numItems > 0);

   // allocate the memory
   int * list = allocate(numItems); // allocated arrays go in pointer variables
   if (list == NULL)
      return 1;

   // do something with it
   fillList(list, numItems);        // always pass the size with the array
   displayList(list, numItems);

   // make like a tree
   delete [] list;                  // never forget to release the memory
   list = NULL;                     // you can say I am a bit paranoid
   return 0;
}
```

See Also

The complete solution is available at 4-1-allocateArray.cpp or:

```
/home/cs124/examples/4-1-allocateArray.cpp
```

Example 4.1 – Expanding Array

Demo

This example will demonstrate how to grow an array to accomidate an unlimited amount of data. This will be accomplished by detecting when the array is full, allocating a new buffer of twice the size as the first, copying the original data to the new buffer, and freeing the original buffer.

Problem

Write a program to read all the data in a file into a single string, then report how much data was read.

```
Filename: 4-1-expandingArray.cpp
reallocating from 4 to 8
reallocating from 8 to 16
reallocating from 16 to 32
reallocating from 32 to 64
reallocating from 64 to 128
reallocating from 128 to 256
reallocating from 256 to 512
reallocating from 512 to 1024
reallocating from 1024 to 2048
reallocating from 2048 to 4096
Total size: 2965
```

Solution

Most of the work is done in the reallocate function. It will double the size of the current buffer.

```
char * reallocate(char * bufferOld, int &size)
{
   cout << "reallocating from " << size << " to " << size * 2 << endl;

   // allocate the new buffer
   char *bufferNew = new(nothrow) char[size *= 2];
   if (NULL == bufferNew)
   {
      cout << "Unable to allocate a buffer of size " << size << endl;
      size /= 2;                                    // reset the size
      return bufferOld;
   }

   // copy the data into the new buffer
   int i;
   for (i = 0; bufferOld[i]; i++)                   // use index because it is easier
      bufferNew[i] = bufferOld[i];                  //   than two pointers
   bufferNew[i] = '\0';                             // don't forget the NULL

   // delete the old buffer
   delete [] bufferOld;

   // return the new buffer
   return bufferNew;
}
```

Challenge

It may seem a bit wasteful to double the size of the buffer with every reallocation. Would it be better to increase the size by 50%, by 200%, or by a fixed amount (say 100 characters)? Modify the above code to accommodate these different strategies and find out which has the smallest amount of wasted space and the smallest number of reallocations.

See Also

The complete solution is available at 4-1-expandingArray.cpp or:

```
/home/cs124/examples/4-1-expandingArray.cpp
```

Example 4.1 – Allocate Images

Demo

Our final example will demonstrate how to allocate the space necessary to display a digital picture. In this example, the user will provide the size of the image; it is not known at compile time. There is one important side-effect from this: we cannot use the multi-dimensional array notation with allocated memory because the compiler must know the size of the array to use that notation. Since the size is not known until run-time, this approach is impossible.

Solution

The code to allocate the image is the following:

```
/*********************************
 * ALLOCATE
 * Grab the memory, returning NULL if
 * anything went wrong
 *********************************/
char * allocate(int numRow, int numCol)
{
   assert(numRow > 0 && numCol > 0);

   // we allocate a 1-dimensional array and do the
   // two dimensional math ourselves
   char * image = new(nothrow) char[numRow * numCol];
   if (!image)
   {
      cout << "Unable to allocate "
           << numRow * numCol * sizeof(char)
           << " bytes for a "
           << numCol << " x " << numRow
           << " image\n";
      return NULL;
   }
   return image;
}
```

Observe how we must work with 1-dimensional arrays even though the image is 2-dimensional. Therefore we must do the transformations ourselves:

```
/*********************************
 * DISPLAY
 * Display the image. This is ASCII-art
 * so it is not exactly "High resolution"
 *********************************/
void display(const char * image, int numRow, int numCol)
{
   // paranoia
   assert(image);
   assert(numRow > 0 && numCol > 0);

   // display the grid
   for (int row = 0; row < numRow; row++)       // two dimensional loop, first
   {                                            //    the rows, then
      for (int col = 0; col < numCol; col++)    //    the columns
         cout << image[row * numCol + col];     // do the [] math ourselves
      cout << endl;
   }
}
```

See Also

The complete solution is available at:

```
/home/cs124/examples/4-1-allocateImages.cpp
```

Problem 1

What is the output of the following code?

```
{
   float * p = NULL;

   cout << sizeof(p) << endl;
}
```

Answer:

_____

Please see page 258 for a hint.

Problem 2

What is the output of the following code?

```
{
   int a[40];

   cout << sizeof(a[42]) << endl
}
```

Answer:

_____

Please see page 215 for a hint.

Problem 3

How much memory does each of the following variables require?

`char text[2]`	
`char text[] = "Software";`	
`int nums[2];`	
`bool values[8];`	

Please see page 215 for a hint.

Problem 4

Write the code to declare a pointer to an integer variable and allocate it.

Answer:

Please see page 327 for a hint.

Problem 5

How do you indicate that you no longer need memory that was previously allocated? Write the code to free the memory pointed to by the variable `p`.

Answer:

Please see page 328 for a hint.

Problem 6

What statement is missing in the following code?

```
{
   float * pNum = new float;

   delete pNum;

   <statement belongs here>
}
```

Answer:

_____

Please see page 325 for a hint.

Problem 7

How much memory is allocated with each of the following?

`p = new double;`	
`p = new char[8];`	
`p = new int[6];`	
`p = new char(65);`	

Please see page 328 for a hint.

Problem 8

Write the code to prompt the user for a number of `float` grades, then allocate an array just big enough to store the array.

Please see page 330 for a hint.

Assignment 4.1

Write a program to:

- Prompt the user for the number of characters in a string
- Allocate a string of sufficient length (one more than # of characters!)
- Prompt the user for the string using `getline`
- Display the string back to the user
- Don't forget to release the memory and check for allocation failures!

Note that since the first `cin` will leave the stream pointer on the newline character, you will need to use `cin.ignore()` before `getline()` to properly fetch the section input.

Examples

Three examples… The user input is **underlined**.

Example 1

```
Number of characters: 13
Enter Text: NoSpacesHere!
Text: NoSpacesHere!
```

Example 2

```
Number of characters: 45
Enter Text: This is a ton of characters. How long is it?
Text: This is a ton of characters. How long is it?
```

Example 3 (allocation failure)

```
Number of characters: -10
Allocation failure!
```

Assignment

The test bed is available at:

```
testBed cs124/assign41 assignment41.cpp
```

Don't forget to submit your assignment with the name "Assignment 41" in the header.

Please see page 330 for a hint.